WEB SECURITY & VULNERABILITY ASSESSMENT AND PENETRATION TESTING (VAPT)

Project Report



Internship Name: DG Interns

Branch: Cyber Security Internship

Prepared by: **Katyayni Negi**

Submission Date: 14th October 2026

Tools Used: Burp Suite (Community Edition), OWASP ZAP, DVWA

Table of Contents

1. Introduction to VAPT 3

1.1 Objectives of the Exercise 3

1.2 Scope 3

2. OWASP Top 10 - Study Notes 4

2.1 SQL Injection (SQLi) 4

2.2 Cross-Site Scripting (XSS) 4

2.3 Broken Authentication 5

2.4 Security Misconfiguration 6

3. Tools Used 6

3.1 Burp Suite (Community Edition) 6

3.2 OWASP ZAP (Zed Attack Proxy) 6

3.3 DVWA (Damn Vulnerable Web Application) 7

4. Testing Process 7

4.1 SQL Injection Testing (DVWA) 8

4.2 Reflected XSS Testing (DVWA) 8

4.3 Stored XSS Testing (DVWA) 9

4.4 Intercepting Requests with Burp Suite 10

5. Automated Scan Using OWASP ZAP 11

5.1 Scanning Process 11

5.2 Key Automated Findings 12

6. Findings & Risk Levels 12

7. Mitigation Methods 13

7.1 For SQL Injection 13

7.2 For Cross-Site Scripting 13

7.3 For Broken Authentication 13

7.4 For Security Misconfiguration 13

8. Conclusion 13

8.1 References 14

1. Introduction to VAPT

Vulnerability Assessment and Penetration Testing (VAPT) is a combination of two distinct security testing approaches used to identify, evaluate, and exploit weaknesses in computer systems, networks, and web applications. A Vulnerability Assessment focuses on discovering and cataloguing known weaknesses using automated scanning tools, while Penetration Testing goes a step further by actively attempting to exploit those weaknesses in a controlled manner to understand the real-world impact of an attack.

In the context of web applications, VAPT is used to uncover flaws such as injection vulnerabilities, broken authentication mechanisms, insecure configurations, and weaknesses in session management before malicious actors can exploit them. Regular VAPT exercises form a core part of any organisation's cybersecurity strategy, helping reduce the attack surface, meet compliance requirements, and build trust with end users.

This report documents a hands-on VAPT exercise performed on the Damn Vulnerable Web Application (DVWA), a deliberately insecure PHP/MySQL application built for practising common web attacks, and on the publicly available test target testphp.vulnweb.com. The exercise covers manual testing of SQL Injection and Cross-Site Scripting (XSS), request interception using Burp Suite, and automated scanning using OWASP ZAP.

1.1 Objectives of the Exercise

- To understand the OWASP Top 10 web application vulnerabilities, with focus on SQL Injection, XSS, Broken Authentication, and Security Misconfiguration.
- To install and configure industry-standard VAPT tools: Burp Suite, OWASP ZAP, and DVWA.
- To manually test and exploit vulnerabilities in a safe, legal, and isolated lab environment.
- To intercept and analyse HTTP requests/responses using a proxy tool (Burp Suite).
- To perform an automated vulnerability scan using OWASP ZAP and interpret the results.
- To document findings with appropriate risk ratings and recommend mitigation strategies.

1.2 Scope

Testing was restricted to the DVWA application hosted locally via XAMPP/Docker, and the publicly permitted test site testphp.vulnweb.com, which is explicitly provided by Acunetix for legal security testing practice. No other systems, networks, or third-party applications were targeted. All testing was carried out in a controlled lab environment for educational purposes only.

2. OWASP Top 10 - Study Notes

The OWASP (Open Web Application Security Project) Top 10 is a globally recognised list of the most critical security risks to web applications. Below are detailed notes on the four vulnerability categories covered in this exercise.

2.1 SQL Injection (SQLi)

SQL Injection is a code injection technique in which an attacker inserts malicious SQL statements into an entry field (such as a login form or search box) for execution by the backend database. It occurs when user-supplied input is concatenated directly into a SQL query without proper validation, sanitisation, or the use of parameterised queries.

How it works

A typical vulnerable query looks like: SELECT * FROM users WHERE username = '" + input + "'. If an attacker enters a value such as ' OR '1'='1, the resulting query always evaluates to true, allowing authentication bypass or unauthorised data retrieval. Attackers may also use UNION-based injection to extract data from other tables, or blind/time-based injection when no direct output is visible.

Impact

- Unauthorised access to sensitive data (usernames, passwords, personal information).
- Authentication bypass without valid credentials.
- Modification or deletion of database records.
- In severe cases, full compromise of the underlying server.

Prevention

- Use parameterised queries / prepared statements instead of string concatenation.
- Apply strict input validation and allow-listing of expected input formats.
- Use ORM frameworks that abstract raw SQL construction.
- Apply the principle of least privilege to database accounts.
- Deploy a Web Application Firewall (WAF) as a defence-in-depth measure.

2.2 Cross-Site Scripting (XSS)

XSS is a client-side injection vulnerability where an attacker injects malicious JavaScript into web pages viewed by other users. Because the browser trusts content served from the vulnerable site, the injected script executes within the victim's session, enabling cookie theft, session hijacking, defacement, or redirection to malicious sites.

Types of XSS

- Reflected XSS: The malicious script is included in the request (e.g., a URL parameter) and immediately reflected back in the response without being stored, usually requiring the victim to click a crafted link.
- Stored XSS: The malicious script is permanently stored on the target server (e.g., in a comment, guestbook, or database field) and served to every user who views the affected page.
- DOM-based XSS: The vulnerability exists in client-side JavaScript that writes attacker-controlled data into the DOM without sanitisation.

Prevention

- Context-aware output encoding/escaping of all user-supplied data before rendering it in HTML, JavaScript, or attribute contexts.
- Implement a strict Content Security Policy (CSP) to restrict script execution sources.
- Use modern frameworks (React, Angular, etc.) that auto-escape output by default.
- Set the HttpOnly and Secure flags on cookies to limit the impact of script-based theft.

2.3 Broken Authentication

Broken Authentication refers to flaws in the mechanisms used to verify user identity and manage sessions. These weaknesses allow attackers to compromise passwords, session tokens, or exploit implementation flaws to assume other users' identities, temporarily or permanently.

Common weaknesses

- Allowing weak, default, or easily guessable passwords without complexity rules.
- No protection against automated brute-force or credential-stuffing attacks (missing rate limiting or account lockout).
- Session IDs exposed in the URL, or not invalidated after logout.
- Missing or weak multi-factor authentication (MFA).
- Predictable or non-expiring session tokens.

Prevention

- Enforce strong password policies and multi-factor authentication.
- Implement account lockout / rate limiting / CAPTCHA against brute-force attempts.
- Use secure, random, and sufficiently long session identifiers; regenerate session IDs after login.
- Invalidate sessions properly on logout and after a defined idle timeout.

2.4 Security Misconfiguration

Security Misconfiguration arises when systems, frameworks, servers, or applications are deployed with insecure default settings, incomplete configurations, verbose error messages, unnecessary features enabled, or missing security headers. It is one of the most common and easily avoidable categories of vulnerability.

Common examples

- Default credentials left unchanged on admin panels or databases.
- Directory listing enabled, exposing file structure to attackers.
- Verbose error messages that reveal stack traces, file paths, or database details.
- Missing security headers such as Content-Security-Policy, X-Frame-Options, and Strict-Transport-Security.
- Outdated software, unpatched frameworks, or unnecessary open ports/services.

Prevention

- Harden server and framework configurations before deployment; disable unused features.
- Regularly apply security patches and updates.
- Disable directory listing and suppress detailed error messages in production.
- Configure recommended HTTP security headers on all responses.
- Perform periodic configuration reviews and automated scans.

3. Tools Used

3.1 Burp Suite (Community Edition)

Burp Suite is an integrated platform for performing web application security testing. The Community Edition used in this exercise provides a man-in-the-middle proxy that intercepts traffic between the browser and the target application, allowing requests and responses to be viewed, modified, and replayed. Key components used include the Proxy (to intercept traffic), Repeater (to resend and modify individual requests), and the built-in HTTP history log.

3.2 OWASP ZAP (Zed Attack Proxy)

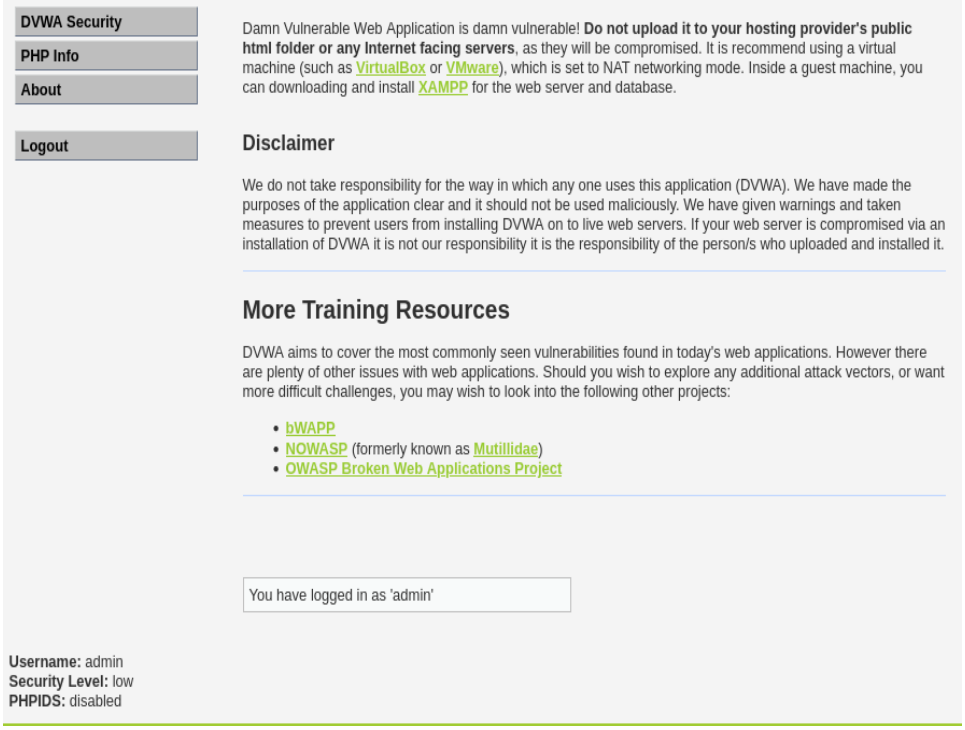
OWASP ZAP is a free, open-source Dynamic Application Security Testing (DAST) tool maintained by the OWASP Foundation. It can perform automated spidering (crawling) of a target application followed by an active scan that tests each discovered endpoint and parameter against a library of known vulnerability signatures, generating a consolidated report of findings.

3.3 DVWA (Damn Vulnerable Web Application)

DVWA is a PHP/MySQL web application that is intentionally vulnerable, designed to help security professionals and students practise common web attacks in a legal environment. DVWA supports configurable security levels (Low, Medium, High, Impossible), allowing the same vulnerability to be tested against progressively stronger defences.

Setup Summary

- Installed XAMPP (Apache + MariaDB + PHP) as the hosting environment (Docker is an equally valid alternative).
- Downloaded the DVWA source code and placed it inside the XAMPP htdocs directory.
- Created the DVWA database and configured config/config.inc.php with database credentials.
- Accessed DVWA via localhost in the browser, logged in with default credentials (admin / password), and set the security level to "Low" for initial testing.



DVWA setup / login page / security level settings]

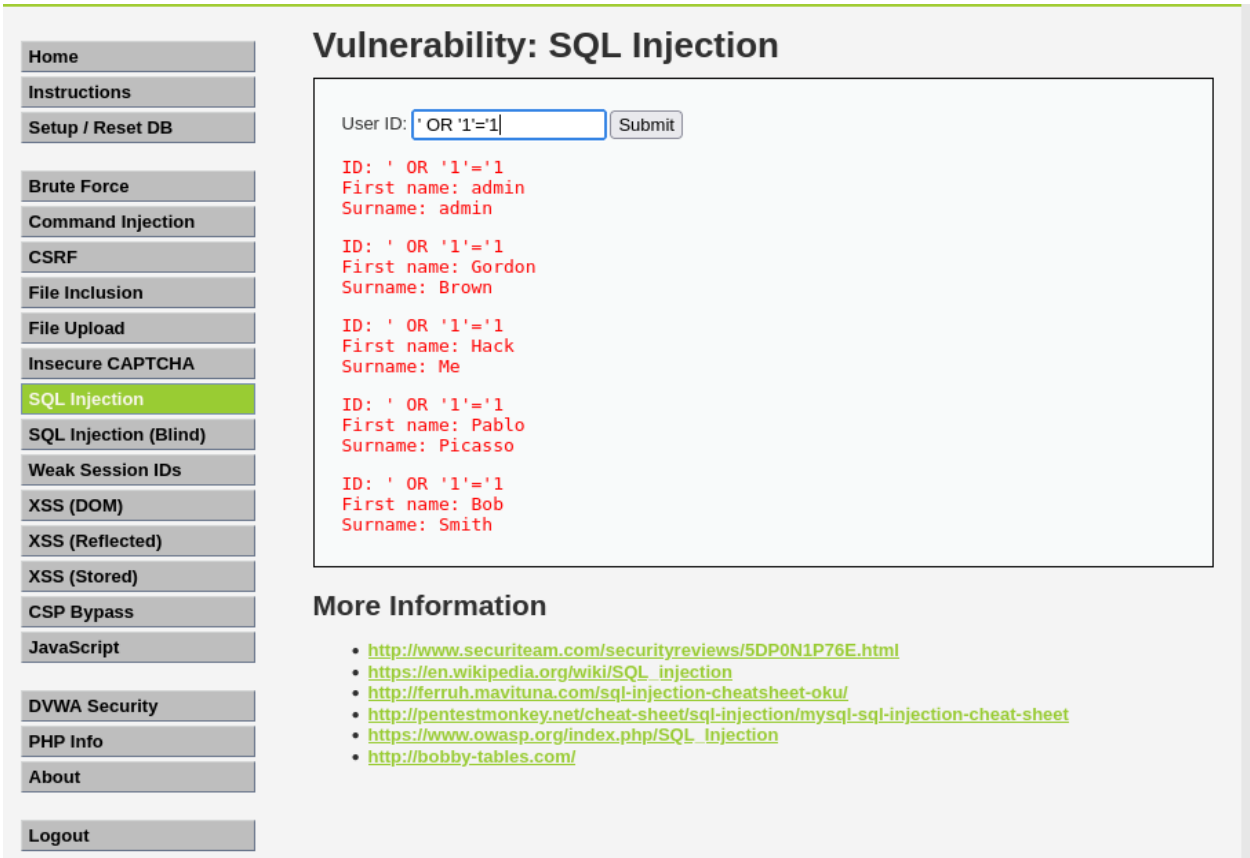
4. Testing Process

All testing described below was performed against DVWA running locally and, for the bonus task, against the publicly available test target testphp.vulnweb.com. The DVWA security level was set to "Low" to allow the underlying vulnerabilities to be observed clearly before being re-tested at higher security levels.

4.1 SQL Injection Testing (DVWA)

The SQL Injection module in DVWA accepts a numeric User ID and queries the backend database for the corresponding user's first and last name.

1. Navigated to the SQL Injection module and entered a normal value (e.g., 1) to observe baseline behaviour.
2. Entered the payload: 1' OR '1'='1 to test whether the input was being concatenated unsafely into the SQL query.
3. Observed that the application returned details for all users in the database, confirming the injection was successful.
4. Used a UNION-based payload, e.g., 1' UNION SELECT user, password FROM users -- , to extract usernames and password hashes from the users table.



SQL Injection payload entered with result in DVWA

4.2 Reflected XSS Testing (DVWA)

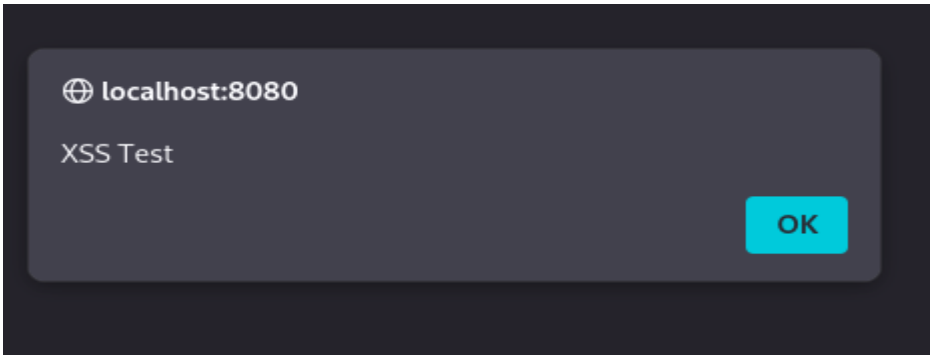
The Reflected XSS module takes a "name" input and echoes it back on the page without sanitisation.

Steps:

1. Entered the payload <script>alert('XSS Test')</script> into the input field.
2. Submitted the form and observed a JavaScript alert box execute immediately, confirming the input was reflected and rendered as executable script.



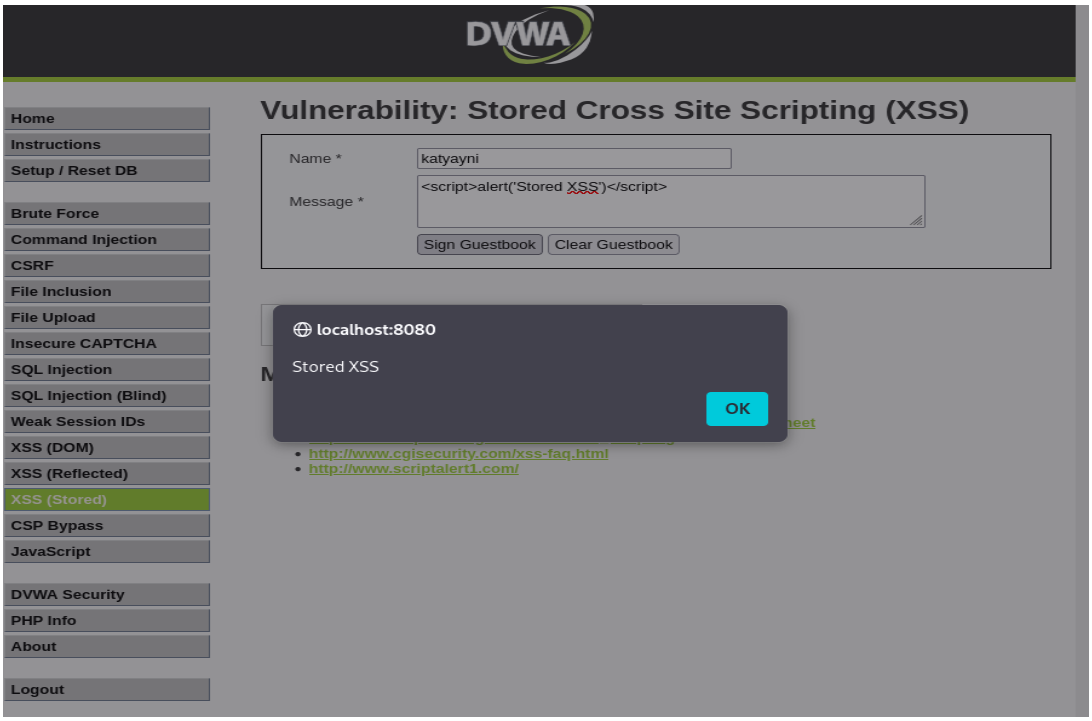
Result:



4.3 Stored XSS Testing (DVWA)

The Stored XSS module (Guestbook) saves user-submitted messages to the database and displays them to every visitor.

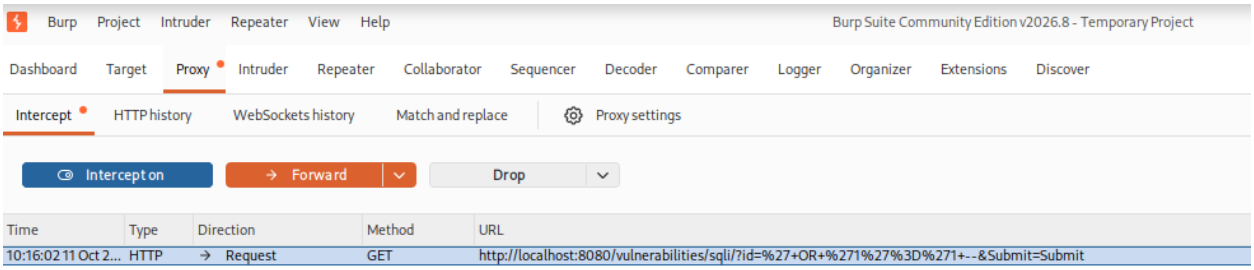
1. Entered the payload <script>alert('Stored XSS')</script> in the message field and submitted the form.
2. Reloaded the page and confirmed the alert box executed automatically, proving the payload was persistently stored and served to all visitors of the page.



4.4 Intercepting Requests with Burp Suite

Burp Suite's proxy was configured as the browser's intermediary (listening on 127.0.0.1:8080) and the browser was set to route traffic through it, with Burp's CA certificate installed for HTTPS interception.

1. Intercepted the login and SQL Injection requests in the Proxy > Intercept tab.
2. Forwarded captured requests to Repeater to modify the "id" parameter directly and resend the request without using the browser.
3. Reviewed the raw HTTP request and response in Burp to confirm how the injected payload was transmitted and reflected by the server.



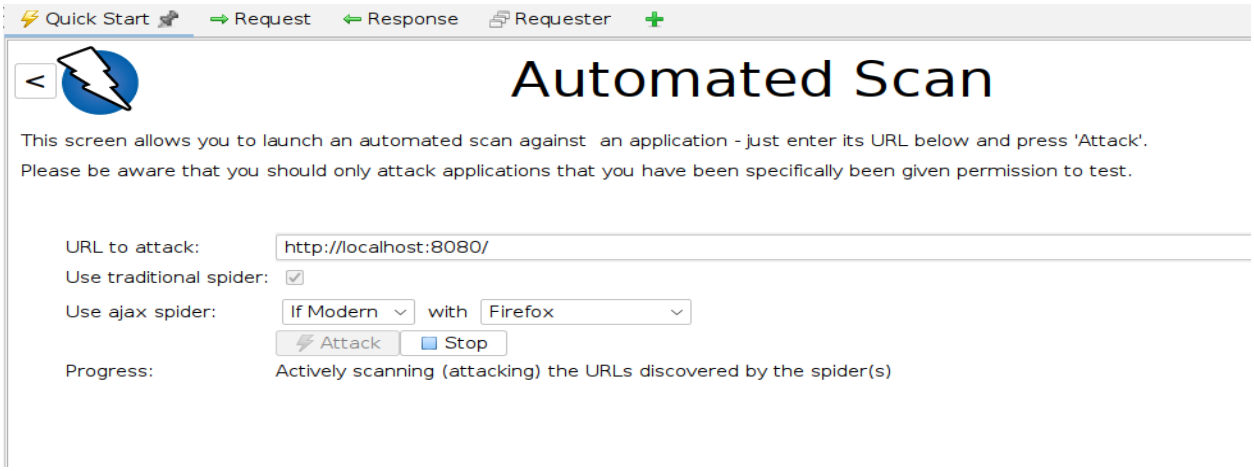
Burp Suite Proxy intercepting the SQL Injection request

5. Automated Scan Using OWASP ZAP

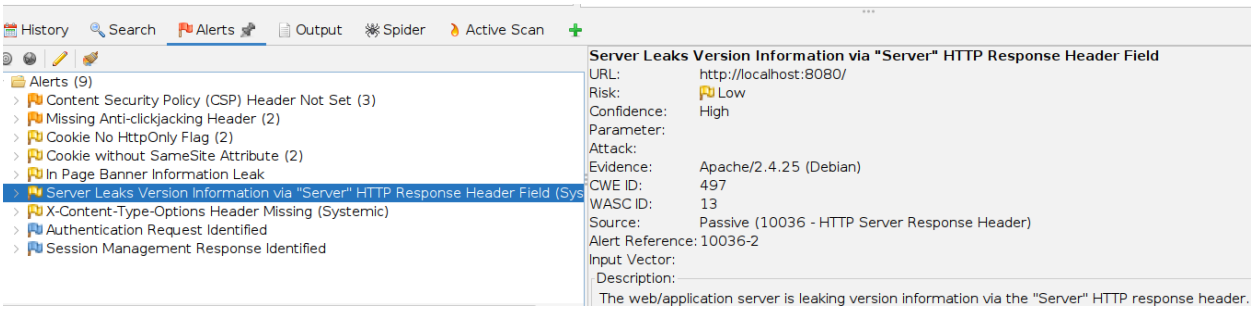
An automated scan was run against the DVWA application (and optionally testphp.vulnweb.com) using OWASP ZAP to cross-validate the manually discovered vulnerabilities and identify any additional issues.

5.1 Scanning Process

1. Added the target URL to ZAP's Sites tree and ran the Spider tool to crawl and enumerate all reachable pages and parameters.
2. Ran an Active Scan against the spidered site, which systematically injects test payloads into every discovered parameter and analyses the responses.
3. Reviewed the Alerts panel, which groups findings by risk level (High, Medium, Low, Informational).
4. Exported the full results as an HTML/PDF scan report for inclusion in project documentation.



OWASP ZAP Spider/Active Scan in progress



OWASP ZAP Alerts panel with identified vulnerabilities

5.2 Key Automated Findings

- SQL Injection - High risk alert raised on the User ID parameter.
- Cross-Site Scripting (Reflected) - High risk alert raised on input fields lacking output encoding.
- Missing Anti-CSRF Tokens - Medium risk alert on form submissions.
- X-Frame-Options Header Not Set - Low risk, enabling potential clickjacking.
- Server leaks version information via the "Server" HTTP response header - Informational/Low risk.

6. Findings & Risk Levels

The table below consolidates all vulnerabilities identified during manual testing (DVWA, testphp.vulnweb.com) and the automated OWASP ZAP scan, along with an approximate CVSS-based risk rating.

#	Vulnerability	Location / Module	Risk Level	CVSS (approx.)
1	SQL Injection (Low security level)	DVWA - SQL Injection module	Critical	9.8
2	Reflected Cross-Site Scripting (XSS)	DVWA - XSS Reflected module	High	7.2
3	Stored Cross-Site Scripting (XSS)	DVWA - XSS Stored (Guestbook)	High	8.0
4	Broken Authentication	DVWA - Login / Brute Force module	High	7.5
5	Security Misconfiguration	DVWA - Server & config headers	Medium	5.8
6	SQL Injection	testphp.vulnweb.com - artists.php (id param)	Critical	9.4
7	Reflected XSS	testphp.vulnweb.com - search.php	High	7.0
8	Missing Security Headers	testphp.vulnweb.com - HTTP response headers	Low	3.5

Risk levels are categorised as Critical, High, Medium, and Low based on exploitability, impact on confidentiality/integrity/availability, and ease of remediation. SQL Injection vulnerabilities were rated Critical due to the potential for full database compromise with minimal attacker effort.

7. Mitigation Methods

7.1 For SQL Injection

- Replace dynamic SQL string concatenation with parameterised queries / prepared statements.
- Apply strict server-side input validation and type checking (e.g., enforce numeric IDs).
- Restrict database account privileges following the principle of least privilege.
- Deploy a Web Application Firewall (WAF) to detect and block common injection patterns.

7.2 For Cross-Site Scripting

- Encode/escape all user-controlled output based on the rendering context (HTML, JS, URL, attribute).
- Implement a strict Content Security Policy (CSP) header to restrict inline/external script execution.
- Validate and sanitise input on both client and server sides; avoid trusting any stored user content.
- Set HttpOnly and Secure flags on session cookies to reduce the impact of a successful XSS attack.

7.3 For Broken Authentication

- Enforce strong password policies and offer/require multi-factor authentication.
- Add account lockout, rate limiting, and CAPTCHA to defend against brute-force attacks.
- Use secure, randomly generated session tokens; regenerate session IDs after successful login.
- Ensure sessions are invalidated on logout and expire automatically after inactivity.

7.4 For Security Misconfiguration

- Change all default credentials immediately after installation of any software or service.
- Disable directory listing and verbose error/debug output in production environments.
- Apply security headers (CSP, X-Frame-Options, X-Content-Type-Options, HSTS) on every response.
- Maintain a regular patching schedule and periodically re-scan the environment for drift.

8. Conclusion

This VAPT exercise provided practical, hands-on experience in identifying and exploiting common web application vulnerabilities using an industry-standard methodology: manual testing against a deliberately vulnerable application (DVWA), traffic interception and analysis with Burp Suite, and automated vulnerability scanning with OWASP ZAP. The exercise successfully

demonstrated SQL Injection, Reflected XSS, and Stored XSS on DVWA, and validated the same vulnerability classes on the public test target testphp.vulnweb.com.

The findings reinforce that a layered defence strategy - combining secure coding practices such as parameterised queries and output encoding, strong authentication controls, hardened server configurations, and routine automated scanning - is essential to reducing the attack surface of any web application. Continuous VAPT exercises, both manual and automated, remain a critical part of a mature application security programme.

8.1 References

- OWASP Foundation - OWASP Top 10 Project (owasp.org).
- DVWA - Damn Vulnerable Web Application documentation (dvwa.co.uk).
- PortSwigger - Burp Suite Community Edition documentation.
- OWASP ZAP official documentation (zaproxy.org).
- Acunetix - testphp.vulnweb.com public test site.